

REFURBNICS

RFID Setup Guide

How to add RFID / NFC battery identification to the Refurbnics platform — hardware, tags, database, backend API, mobile app, web app, rollout and testing.

Item	Value
Project	Refurbnics battery repair platform (backend / frontend / mobile)
Document	RFID.pdf — generated from the repository state on 5 Oct 2026 (commit 0ecaf21)
Audience	Developer implementing the feature; workshop manager buying hardware
Companion files	RFID.md (short plan), cerebrum.md (project brain — read §3.2, §7, §8 first)
Live systems	API https://refurbincsuk.onrender.com (Render + Neon) · Web on Vercel · Android APK via EAS

Contents

1. Summary — what RFID adds and why it is a small change here
 2. How battery scanning works today (the one thing RFID must produce)
 3. Choose a technology: NFC, UHF handheld, or fixed UHF gates
 4. Hardware: what to buy, what to avoid, costs
 5. Tag content strategy (URL on tag + UID in database)
 6. Step-by-step implementation: database → backend → mobile → web
 7. Programming (encoding) tags for a client fleet
 8. Daily use: technician, client, admin flows with RFID
 9. Testing checklist
 10. Rollout plan and effort
 11. Costs
 12. Risks, limits and FAQ
- Appendix A — Full code listings
- Appendix B — Files to touch (checklist)

1. Summary

RFID lets a technician, supervisor or client identify a battery by **tapping or reading a tag** instead of pointing a camera at a QR code. In this project that is a small addition, because every scan path already ends in the same place: a plain **battery code string** (for example HUM-0000123) which is sent to GET /api/batteries/:code. An RFID reader only has to produce that same string, or a tag ID the backend can map to it. Nothing downstream — start work, testing, pack to repair, intake verify, returns — needs to change.

What changes	Size
Database: one new column batteries.rfid_tag (migration 054)	5 lines of SQL
Backend: battery lookup matches code or tag; one endpoint to assign a tag to a battery	~60 lines
Mobile app: NFC read hook + a 'Tap RFID' button on the scan screens + an admin 'Program tag' screen	2–3 days
Web app: accept Tab as well as Enter from keyboard-mode readers; show/assign the tag on the battery page	½ day
Hardware: on-metal NFC tags (£0.20–0.50 each) and, optionally, a USB desk reader (~£30) or UHF gun (£400+)	1 week lead time

Tip: Start with NFC stickers and the phones you already use (Option A in §3). It costs about £40 to pilot and needs no new reader hardware for the mobile app. Move to UHF only if you need to read many batteries at a distance.

2. How scanning works today

All three scan inputs converge on one string and one API call:

```
camera QR (expo-camera / QrScanner.jsx) ■
USB / Bluetooth scanner gun (types text) ■■■ extractBatteryCode(raw) ■■ GET /api/batteries/:code
typed code + Enter ■ frontend/src/utils/extract-battery-code.js
■■
start-work · start/complete-testing · report-issue · pack-to-repair · verify-arrival ·
returns · recycle
```

Where	File	How the code arrives
Technician web (scan page)	frontend/src/features/batteries/technician/TechnicianHomePage.jsx	QrScanner.jsx camera, or an input with handleManualKeyDown (Enter)
Technician mobile	mobile/src/screens/technician/ServiceScreen.js	expo-camera CameraView onBarcodeScanned (debounced)
Client mobile scan / sorting	mobile/src/screens/client/ClientScanScreen.js, ClientSortingScreen.js	expo-camera
Client web pack-to-repair	frontend/src/features/clients/portal/ClientBatteriesPage.jsx	typed code with suggestions
Admin truck verify / returns / recycle	TruckVerifyModal.jsx, ReturnForm.jsx, RecycleForm.jsx	window keydown listener (Enter) / inputs
Backend lookup	backend/src/models/battery.model.js → findByCode()	WHERE b.battery_code = \$1

The QR code printed on each battery encodes the public URL `https://<web>/batteries/HUM-0000123`; `extractBatteryCode()` strips the URL and returns the code. Battery codes are the client's 3-letter prefix plus a 7-digit zero-padded number (older rows have 4 digits, e.g. HUM-0042; both are valid).

3. Choose a technology

Option	Hardware	How it reaches the app	Code change	Best for
A. NFC (13.56 MHz) start here	On-metal NFC stickers (NTAG213 / NTAG216) + phone NFC (Android; iPhone 7+) or a USB desk reader (ACR122U)	Mobile: react-native-nfc-manager reads the tag. Web: USB reader acts as a keyboard and types the code + Enter	Small	Bench work, one battery at a time, 2–4 cm range (same as QR)
B. UHF handheld (860–960 MHz)	On-metal UHF tags + handheld gun (Zebra RFD40, Chainway C72, Urovo)	Pairs as a Bluetooth keyboard (HID) and types each tag ID + Enter into any focused input	Small / none	Reading 1–5 m away, many batteries fast (truck intake, returns count)
C. Fixed UHF gate	UHF tags + fixed reader (Impinj R700, Zebra FX9600) + antennas at the door / loading bay	Reader posts tag reads to a new backend webhook	Medium	Automatic intake / dispatch while a truck unloads, no human scan

Recommended path: **A** → **B** → **C**. A is cheap and uses the Android phones technicians already hold. B is a hardware-only upgrade (no code) because the gun types like a keyboard. C is a later automation step.

Frequency and range

	NFC / HF 13.56 MHz	UHF 860–960 MHz
Read range	0–4 cm (tap)	0.5–8 m depending on tag and antenna
Reads many tags at once	No (one tap = one tag)	Yes (dozens per second)
Works with a phone	Yes, built-in	No — needs a gun or sled
On-metal tags available	Yes	Yes (better selection)
Typical tag cost (on-metal)	£0.20–0.50	£0.30–1.50
Readers	Phone £0 · USB desk reader ~£30	Handheld £400–1,500 · fixed £1,000–3,000

4. Hardware

4.1 Tags — the one rule

Note: Batteries are metal and dense. **Ordinary RFID/NFC stickers will not read on them.** Buy tags sold as **on-metal / anti-metal** (they have a ferrite layer). Test 5–10 on a real battery before a bulk order.

Type	Look for	Notes
NFC on-metal sticker	NTAG213 on-metal (144 bytes) or NTAG216 anti-metal (888 bytes); 25–30 mm round or 20×40 mm	Enough memory for the battery URL. Printable label surface so the code is also human-readable
NFC rugged disc	Epoxy or ABS on-metal disc, 20–30 mm, adhesive or screw-mount	Survives heat and handling better than paper stickers
UHF on-metal	Confidex Ironside Micro, Omni-ID Fit, Xerafy Dot-on	Only if you choose Option B/C

4.2 Readers

Reader	Use	Cost	Notes
Android phone with NFC	Mobile app (technician & client)	£0	Most Android phones since 2015. Check Settings → Connections → NFC
iPhone 7 or newer	Mobile app read (iOS 13+ can also write)	£0	iOS shows a system sheet for every read — slower for bulk
ACR122U USB NFC reader	Admin desk: assign tags, verify intake on the web	~£30	Needs a small helper app or the vendor's keyboard-emulation tool to type the UID
NFC keyboard-emulation reader (e.g. uTrust 3700 F, 'NFC HID reader')	Web pages without extra software	£40–80	Types the tag UID or NDEF text + Enter — works in any input
UHF handheld (Zebra RFD40 sled, Chainway C72, Urovo DT50U)	Bulk reads at intake / returns	£400–1,500	Use Bluetooth HID (keyboard) mode; set suffix to Enter
Fixed UHF reader + 2–4 antennas	Door / loading-bay gates	£1,000–3,000	Option C only. Needs a network connection to post to the API

4.3 Tag placement

- Same spot on every battery, flat and not recessed, so staff know where to tap.
- Away from terminals and from the area the technician opens during repair.
- Clean the surface with alcohol first; press firmly for 10 seconds; let the adhesive cure 24 h before heat.
- Print the battery code on the tag label as a fallback (the QR label can stay next to it).

4.4 Starter kit (~£40)

- 20 × NTAG213 on-metal stickers (25 mm)
- 1 × ACR122U or an NFC keyboard-emulation reader for the admin desk
- Any Android phone with NFC for the technician test

5. Tag content strategy

Decide this once. The recommendation is to **do both**:

Strategy	Pros	Cons
Write the battery URL onto the tag as an NDEF record (same content as the QR: <code>https://.../batteries/HUM-0000123</code>)	Zero backend change; any NFC phone — even without the app — opens the battery's public page; <code>extractBatteryCode()</code> already understands it	Tag can be rewritten unless locked
Store the tag's factory UID / EPC in <code>batteries.rfid_tag</code>	Cannot be cloned or edited; UHF tags (Options B/C) only ever report their EPC, so this is required for them	Every tag must be registered to a battery first (the 'Program tag' step)

With both in place: an Android phone reads the NDEF URL and the flow is identical to a QR scan; a UHF gun or an unlocked/blank tag falls back to the UID, which the backend resolves through `rfid_tag`. Lock NFC tags after writing (`NfcManager.ndefHandler.makeReadOnly()`) so a tag cannot be re-pointed at another battery.

Tip: Store the UID upper-cased without separators (e.g. 04A2B3C4D5E680). Readers report it in different formats (colons, spaces, lower-case); normalise once in the backend.

6. Step-by-step implementation

Order matters: database → backend → (deploy) → mobile → web. Each step is independently deployable; after step 2 any reader that types the UID already works everywhere.

Step 1 — Database (migration 054)

Migrations run automatically when the backend starts (Render redeploy or `npm run dev`). Numbering: the last file is `053_issue_failed_testing.sql`, so this one is 054.

```
-- backend/src/db/migrations/054_battery_rfid_tag.sql
-- Links a physical RFID/NFC tag (factory UID or UHF EPC, normalised upper-case
-- hex, no separators) to a battery so a tag read resolves exactly like a QR scan.
ALTER TABLE batteries ADD COLUMN IF NOT EXISTS rfid_tag VARCHAR(64) UNIQUE;
CREATE INDEX IF NOT EXISTS idx_batteries_rfid_tag ON batteries (rfid_tag);
```

Step 2 — Backend

2a. Lookup by code or tag

In `backend/src/models/battery.model.js`, `findByCode()` currently ends with `WHERE b.battery_code = $1`. Change it so a tag ID also matches:

```
// battery.model.js – findByCode(): replace the final WHERE
WHERE b.battery_code = $1
OR b.rfid_tag = upper(regexp_replace($1, '[^0-9A-Za-z]', '', 'g'))`,
```

This single change makes every existing endpoint that calls `findByCode` (public lookup, start-work, testing, report-issue, client pack, truck verify, returns) accept a tag ID. Also add `b.rfid_tag` to the `SELECT` lists in `findPage` so the fleet table and detail page can show it (`findByCode` already uses `b.*`).

2b. Assign / replace a tag

New route in `backend/src/routes/battery.routes.js` (office roles only — the same pattern as `PATCH /:id/client`):

```
// battery.routes.js
router.patch('/:id/rfid-tag', requireRole('super_admin', 'admin', 'staff'),
  batteryController.updateRfidTag);

// battery.controller.js
async function updateRfidTag(req, res, next) {
  try {
    const raw = typeof req.body.rfidTag === 'string' ? req.body.rfidTag : '';
    const rfidTag = raw.replace(/[^0-9A-Za-z]/g, '').toUpperCase() || null; // null = remove tag
    if (rfidTag && (rfidTag.length < 8 || rfidTag.length > 64)) {
      return res.status(400).json({ message: 'Tag ID must be 8-64 hex characters.' });
    }
    const battery = await batteryModel.updateRfidTag(req.params.id, rfidTag);
    if (!battery) return res.status(404).json({ message: 'Battery not found' });
    await auditLogModel.record({
      userId: req.user.id, action: 'assign_rfid_tag', entity: 'battery', entityId: battery.id,
      details: { batteryCode: battery.battery_code, rfidTag },
    });
    realtime.broadcastBatteryUpdated(battery);
    res.json(battery);
  } catch (err) {
    if (err.code === '23505') {
      return res.status(409).json({ message: 'This tag is already assigned to another battery.' });
    }
    next(err);
  }
}

// battery.model.js
async function updateRfidTag(id, rfidTag) {
  const { rows } = await db.query(
    'UPDATE batteries SET rfid_tag = $2 WHERE id = $1 RETURNING *',
```

```

    [id, rfidTag]
  );
  return rows[0];
}

```

Add `updateRfidTag` to both `module.exports`. Also add a lookup for the assign screen so the admin can type a tag and see which battery owns it: `GET /api/batteries/by-tag/:tag` (workshop roles) → `findByCode(tag)`, 404 if none. Register it **above** `/:code` in the router or it will be swallowed as a code.

2c. Option C only — gate reader webhook

```

POST /api/rfid/events      Authorization: Bearer <per-reader API key>
{ "readerId": "intake_gate_1", "tagId": "E2801160600002...", "antenna": 2, "ts":
  "2026-10-05T10:21:00Z" }

```

- New table `rfid_readers` (`id`, `name`, `api_key_hash`, `action`, `client_id`, `active`); the key is checked by a small middleware, not the user JWT.
- `action` maps a reader to a flow: `intake_gate` → add the battery to today's open intake for that client and mark arrival; `dispatch_gate` → add it to the open return.
- Debounce: ignore the same tag from the same reader within 30 s (gates read a tag many times while it passes).
- Never change status automatically on a read that contradicts the state machine (e.g. a battery in testing passing the dispatch gate) — log it as an alert instead.

Step 3 — Mobile app (Expo 57)

Note: NFC is a native module. It is **not** available in Expo Go, but this project already ships as an EAS build (`mobile/eas.json`, Android package `com.refurbbinics.technician`), so you only need to install the package and rebuild the APK.

3a. Install and configure

```

cd mobile
npx expo install react-native-nfc-manager

// mobile/app.json (merge into the existing "expo" object)
"plugins": [
  ["expo-camera", { "cameraPermission": "Refurbbinics needs camera access to scan battery QR codes."
  }],
  "expo-sharing",
  "expo-status-bar",
  ["react-native-nfc-manager", {
    "nfcPermission": "Refurbbinics uses NFC to read battery RFID tags.",
    "selectIdentifiers": [],
    "includeNdefEntitlement": true
  }]
],
"android": { "permissions": ["android.permission.CAMERA", "android.permission.RECORD_AUDIO",
  "android.permission.NFC"], ... },
"ios": { "infoPlist": { "NFCReaderUsageDescription": "Scan battery RFID tags" }, ... }

```

3b. Read hook

```

// mobile/src/utils/useNfcRead.js
import { useCallback, useEffect, useRef, useState } from 'react';
import NfcManager, { NfcTech, Ndef } from 'react-native-nfc-manager';

function normaliseUid(id) {
  return String(id || '').replace(/^[^0-9A-Za-z]/g, '').toUpperCase();
}

// Returns { supported, enabled, reading, readOnce }.
// readOnce(onCode) opens the NFC session, waits for one tag and calls onCode with
// the NDEF text/URL if present, otherwise the tag UID – the same string the camera
// path hands to extractBatteryCode().
export default function useNfcRead() {

```

```

const [supported, setSupported] = useState(false);
const [enabled, setEnabled] = useState(false);
const [reading, setReading] = useState(false);
const inFlight = useRef(false);

useEffect(() => {
  let cancelled = false;
  NfcManager.isSupported().then(async (ok) => {
    if (cancelled) return;
    setSupported(ok);
    if (ok) { await NfcManager.start(); setEnabled(await NfcManager.isEnabled()); }
  });
  return () => { cancelled = true; NfcManager.cancelTechnologyRequest().catch(() => {}); };
}, []);

const readOnce = useCallback(async (onCode) => {
  if (inFlight.current) return; // one QR-style in-flight lock (cerebrum §7 #27)
  inFlight.current = true;
  setReading(true);
  try {
    await NfcManager.requestTechnology(NfcTech.Ndef, { alertMessage: 'Hold the phone on the battery tag' });
    const tag = await NfcManager.getTag();
    const record = tag?.ndefMessage?.[0];
    let text = null;
    if (record) {
      try { text = Ndef.uri.decodePayload(record.payload); } catch { /* not a URI record */ }
      if (!text) { try { text = Ndef.text.decodePayload(record.payload); } catch { /* not text */ } }
    }
    onCode(text || normaliseUid(tag?.id));
  } finally {
    NfcManager.cancelTechnologyRequest().catch(() => {});
    setReading(false);
    inFlight.current = false;
  }
}, []);

return { supported, enabled, reading, readOnce };
}

```

3c. 'Tap RFID' button on every scan screen

Add the button next to the camera control in `ServiceScreen.js`, `ClientScanScreen.js`, `ClientSortingScreen.js` and the scan entry of `shared/BatteryDetailScreen.js`. It calls the same handler the camera already uses, so debouncing, navigation and error handling stay shared:

```

const { supported, enabled, reading, readOnce } = useNfcRead();
...
{supported && (
  <TouchableOpacity
    disabled={!enabled || reading}
    onPress={() => readOnce((raw) => handleScanned(extractBatteryCode(raw)))}
    className="rounded-2xl bg-violet-600 px-4 py-3 items-center">
    <Text className="text-white font-bold">{reading ? 'Hold phone on tag...' : 'Tap RFID tag'}</Text>
  </TouchableOpacity>
)}
{supported && !enabled && <Text className="text-xs text-rose-600">NFC is turned off – enable it in phone settings.</Text>}

```

Android reads happen silently; iOS pops a system sheet per read (that is the platform behaviour, not a bug).

3d. Admin 'Program tag' screen

A small screen (admin / staff only) that: (1) looks up the battery by code or camera scan, (2) writes the battery URL to the tag, (3) saves the UID through `PATCH /batteries/:id/rfid-tag`, (4) locks the tag. Full listing in Appendix A.

3e. Build

```
cd mobile
```

Step 4 — Web app

- **Keyboard-mode readers** (USB NFC reader, UHF gun over Bluetooth) type the ID then a suffix. Pages already accept Enter in TechnicianHomePage.handleManualKeyDown, TruckVerifyModal, ReturnForm, RecycleForm, ClientBatterySortPage. Change each check from `e.key === 'Enter'` to `(e.key === 'Enter' || e.key === 'Tab')` and call `e.preventDefault()` on Tab so focus does not jump. Many guns default to Tab.
- **Fast typists vs. guns:** a gun types ~20 characters in under 50 ms. If a page debounces input, make sure the debounce does not swallow the Enter.
- **Web NFC** (Android Chrome only): optional `useWebNfc()` hook next to `components/ui/primitives/QrScanner.jsx` using new `NDEFReader().scan()`; show the button only when 'NDEFReader' in window. Desktop browsers have no NFC API — use a keyboard-mode reader.
- **Battery detail page** (`BatteryDetailPage.jsx`): show `rfid_tag` in the identity card; for office roles add 'Assign / replace tag' → focus an input (the USB reader types into it) → PATCH `/batteries/:id/rfid-tag`. Show the 409 message when the tag belongs to another battery.
- **Fleet table** (`BatteriesPage.jsx`): optional 'Tag' column and a 'No tag' filter so the admin can see which batteries still need programming.

```
// Web NFC (Android Chrome) – frontend/src/utils/use-web-nfc.js
export function webNfcSupported() { return typeof window !== 'undefined' && 'NDEFReader' in window;
}
export async function readWebNfcOnce(onCode) {
  const reader = new window.NDEFReader();
  const ctrl = new AbortController();
  await reader.scan({ signal: ctrl.signal });
  reader.onreading = ({ message, serialNumber }) => {
    const rec = message.records[0];
    const text = rec ? new TextDecoder().decode(rec.data) : '';
    ctrl.abort();
    onCode(text || serialNumber.replace(/^[0-9A-Za-z]/g, '').toUpperCase());
  };
}
```

Step 5 — Documentation

- cerebrum.md §3.2: `findByCode` now also matches `rfid_tag`; §3.3 table list; a 'Where to look for X' row for tag problems; fix log entry.
- RFID.md: mark steps done.

7. Programming tags for a client fleet

Each tag has to be written once and linked to a battery. Two ways:

Method	Steps	Speed
Phone (Program tag screen)	1. Open Program tag. 2. Scan the battery QR (or type the code). 3. Hold the phone on the tag → app writes the URL, saves the UID, locks the tag. 4. Green tick → next battery.	~10 s per battery
Desk (USB reader + web)	1. Open the battery page. 2. Click 'Assign tag'. 3. Tap the tag on the USB reader → UID typed into the field → saved. (Tag stays blank; lookup works by UID.) Pre-written URL tags can be ordered from the tag supplier if you send them the code list.	~6 s per battery
Bulk from the supplier	Export battery codes (fleet table → CSV). Supplier encodes URL + prints code on each tag and returns a CSV of code → UID. Import with a one-off script calling PATCH <code>/batteries/:id/rfid-tag</code> .	Best for 500+ batteries

Order of operations for a **new** bulk-generated fleet: Generate QR Code page (bulk) → print QR labels → program tags → apply both labels to the batteries at the client site.

8. Daily use with RFID

Who	Today (QR)	With RFID
Technician starts work	Open Service tab → camera → aim at QR	Open Service tab → 'Tap RFID' → touch phone to battery → same repair panel opens
Supervisor testing	Camera scan of each battery	Tap each battery; or UHF gun from the bench reads the whole shelf into the list
Client packs batteries for repair	Type / scan codes in Pack to Repair	Tap each battery with the phone, or let a UHF gun type the codes into the web form
Admin verifies truck arrival	Scan each QR in Truck Verify	UHF gun in keyboard mode reads all batteries on the pallet; each ID is typed + Enter into the verify modal
Returns dispatch	Scan each battery into the return	Same gun flow; or Option C gate reads automatically as the pallet leaves
Anyone with an NFC phone, no app	Camera app on the QR → public battery page	Tap the tag → phone opens the same public page (NDEF URL)

Permissions are unchanged: a tag read is just a code, so the same role rules in cerebrum.md §3.1 apply (a client can only pack batteries that belong to them — `assertClientCanPack()` still runs).

9. Testing checklist

Hardware pilot

- Tag reads on a real battery at the chosen spot from an Android phone: yes / no, and from how far.
- Tag still reads after one full repair + testing cycle (heat, handling, cleaning).
- Adhesive holds after 1 week.
- Two batteries 10 cm apart: the phone reads the intended one, not its neighbour.

Backend

```
# with a minted JWT (see cerebrum.md §1) against the local API
curl -H "Authorization: Bearer $ADMIN" -X PATCH -H 'Content-Type: application/json' \
  -d '{"rfidTag":"04:a2:b3:c4:d5:e6:80"}' http://localhost:5000/api/batteries/501/rfid-tag #
    200, tag stored as 04A2B3C4D5E680
curl http://localhost:5000/api/batteries/04A2B3C4D5E680 # 200,
    same battery as /HUM-00000001
curl -H "Authorization: Bearer $ADMIN" -X PATCH -d '{"rfidTag":"04A2B3C4D5E680"}'
  .../batteries/502/rfid-tag # 409 already assigned
curl -H "Authorization: Bearer $CLIENT" -X PATCH -d '{"rfidTag":"AA11BB22CC33"}'
  .../batteries/501/rfid-tag # 403
```

Mobile

- Phone with NFC off → button shows the 'NFC is turned off' message, no crash.
- Phone without NFC → button hidden.
- Tap a programmed tag → repair panel for the right battery; tap an unknown tag → the existing 'Battery not found' message.
- Tap twice quickly → one navigation (in-flight lock).
- Program tag on an already-locked tag → clear error, nothing saved.

Web

- USB reader types UID + Enter into Technician scan page → battery opens.
- Gun configured with Tab suffix → still opens the battery, focus does not move.
- Assign a tag already used elsewhere → 409 message shown inline.

10. Rollout plan and effort

Step	Work	Effort
1	Order 20 on-metal NFC tags + 1 USB reader; test range and survival on a battery	1 week lead time
2	Migration 054 + findByCode OR-match + return rfid_tag + assign endpoint + by-tag lookup	½ day
3	BatteryDetailPage tag display + assign action; Tab suffix on web inputs	½ day
4	Mobile: install nfc-manager, useNfcRead hook, 'Tap RFID' buttons, Program-tag screen, EAS build	2–3 days
5	Pilot on one client's fleet (20–50 batteries); compare scan time and error rate against QR	2 weeks
6	If intake / returns need bulk reads → UHF handheld in keyboard mode (no code change)	hardware only
7	Later: fixed gate readers + /api/rfid/events webhook (Option C)	1–2 weeks

11. Costs (indicative, GBP, Oct 2026)

Item	Unit	Pilot (20 batteries)	Fleet of 500
On-metal NFC tags	£0.20–0.50	£10	£100–250
USB NFC desk reader	£30–80	£30	£30–80 (one per desk)
Android phones	existing	£0	£0
UHF on-metal tags (Option B/C)	£0.30–1.50	—	£150–750
UHF handheld gun (Option B)	£400–1,500	—	£400–1,500
Fixed UHF reader + antennas (Option C)	£1,000–3,000 per gate	—	per gate
Developer time (Options A + web)	—	3–4 days	3–4 days

12. Risks, limits and FAQ

Risk / question	Answer
Tags fall off or stop reading after repair	Pilot first (\$9). Prefer epoxy on-metal discs over paper stickers for batteries that get hot.
Does RFID replace QR?	No — keep both. QR needs no special hardware and is the fallback when a tag is damaged. Both resolve to the same code.
iPhone experience	iOS shows a system NFC sheet for each read and cannot scan in the background; fine for one battery at a time, not for bulk. Use Android or a UHF gun for bulk.
Web NFC	Android Chrome only. Desktop web must use a USB / Bluetooth keyboard-mode reader.
Can a battery have more than one tag?	Current design: one rfid_tag per battery; assigning a new one replaces the old. If you need history, add an battery_rfid_tags table instead of the column.

Risk / question	Answer
Tag cloned or moved to another battery	Lock NFC tags after writing; the UID of a genuine NTAG cannot be changed. UHF EPCs can be rewritten — rely on the TID (factory serial) for UHF if tamper resistance matters.
Reader types into the wrong field	Keyboard-mode readers type wherever the cursor is. Scan pages should auto-focus the code input and re-focus after each submit.
Does this affect billing or statuses?	No. A tag read is identical to a QR read; all state changes still go through the same guarded endpoints.
Expo Go	NFC does not work in Expo Go. Use the EAS build (already the release process).

Appendix A — Full code listings

A.1 Program-tag screen (mobile, admin/staff)

```
// mobile/src/screens/admin/ProgramTagScreen.js
import { useState } from 'react';
import { View, Text, TextInput, TouchableOpacity, Alert } from 'react-native';
import NfcManager, { NfcTech, Ndef } from 'react-native-nfc-manager';
import apiClient from '../../services/api-client';
import { extractBatteryCode } from '../../utils/extract-battery-code';

const WEB_ORIGIN = 'https://refurbincs-uk.vercel.app'; // the public web app (QR codes use the
  same origin)

function normaliseUid(id) { return String(id || '').replace(/[^0-9A-Za-z]/g, '').toUpperCase(); }

export default function ProgramTagScreen() {
  const [code, setCode] = useState('');
  const [battery, setBattery] = useState(null);
  const [busy, setBusy] = useState(false);
  const [done, setDone] = useState(null);

  async function lookup() {
    setDone(null);
    const clean = extractBatteryCode(code);
    const { data } = await apiClient.get(`/batteries/${encodeURIComponent(clean)}`);
    setBattery(data.battery);
  }

  async function program() {
    if (!battery) return;
    setBusy(true);
    try {
      await NfcManager.requestTechnology(NfcTech.Ndef, { alertMessage: 'Hold the phone on the new
        tag' });
      const tag = await NfcManager.getTag();
      const uid = normaliseUid(tag.id);
      // 1. write the same URL the QR code carries
      const bytes =
        Ndef.encodeMessage([Ndef.uriRecord(`${WEB_ORIGIN}/batteries/${battery.battery_code}`)]);
      await NfcManager.ndefHandler.writeNdefMessage(bytes);
      // 2. link the UID to the battery (409 if the tag is already used)
      await apiClient.patch(`/batteries/${battery.id}/rfid-tag`, { rfidTag: uid });
      // 3. lock so nobody can re-point the tag
      await NfcManager.ndefHandler.makeReadOnly();
      setDone(uid);
    } catch (err) {
      Alert.alert('Programming failed', err.response?.data?.message || err.message);
    } finally {
      NfcManager.cancelTechnologyRequest().catch(() => {});
      setBusy(false);
    }
  }
}

return (
  <View className="flex-1 bg-slate-50 p-5 gap-4">
    <Text className="text-xl font-black text-slate-900">Program RFID tag</Text>
    <TextInput value={code} onChangeText={setCode} placeholder="Battery code or scan QR"
      autoCapitalize="characters" className="rounded-2xl border border-slate-300 bg-white px-4
        py-3" />
    <TouchableOpacity onPress={lookup} className="rounded-2xl bg-blue-600 py-3 items-center">
      <Text className="text-white font-bold">Find battery</Text>
    </TouchableOpacity>
    {battery && (
      <View className="rounded-2xl border border-slate-200 bg-white p-4 gap-2">
        <Text className="font-extrabold text-slate-900">{battery.battery_code}</Text>
        <Text className="text-xs text-slate-500">{battery.client_name} · current tag:
          {battery.rfid_tag || 'none'}</Text>
        <TouchableOpacity disabled={busy} onPress={program} className="rounded-2xl bg-violet-600
          py-3 items-center">
          <Text className="text-white font-bold">{busy ? 'Hold phone on tag...' : battery.rfid_tag ?
```

```

        'Replace tag' : 'Program tag']</Text>
      </TouchableOpacity>
      {done && <Text className="text-emerald-700 font-bold">Tag {done} linked and
        locked.</Text>}
    </View>
  )}
</View>
);
}

```

A.2 Bulk import of supplier CSV (code,uid) — one-off script

```

// backend/scripts/import-rfid-tags.js  usage: node scripts/import-rfid-tags.js tags.csv
const fs = require('fs');
const db = require('../src/config/db');
(async () => {
  const lines = fs.readFileSync(process.argv[2], 'utf8').trim().split(/\r?\n/).slice(1);
  let ok = 0, missing = 0, dup = 0;
  for (const line of lines) {
    const [code, rawUid] = line.split(',').map((s) => s.trim());
    const uid = rawUid.replace(/[^0-9A-Za-z]/g, '').toUpperCase();
    try {
      const { rowCount } = await db.query(
        'UPDATE batteries SET rfid_tag = $2 WHERE upper(battery_code) = upper($1)', [code, uid]);
      rowCount ? ok++ : missing++;
    } catch (e) { if (e.code === '23505') dup++; else throw e; }
  }
  console.log({ ok, missing, dup });
  process.exit(0);
})();

```

A.3 Option C webhook skeleton

```

// backend/src/routes/rfid.routes.js
const router = require('express').Router();
const { requireReaderKey } = require('../middlewares/reader-key'); // hashes & matches
  rfid_readers.api_key_hash
const rfidController = require('../controllers/rfid.controller');
router.post('/events', requireReaderKey, rfidController.ingest);
module.exports = router;

// rfid.controller.js – ingest(): normalise tagId → findByCode(tagId) → debounce (reader,tag,30s)
// → switch (req.reader.action) { intake_gate: mark arrival on the client's open intake;
//                               dispatch_gate: add to open return } →
  realtime.broadcastBatteryUpdated
// Unknown tag or state conflict → insert into rfid_alerts, respond 202.

```

Appendix B — Files to touch

Layer	File	Change
DB	backend/src/db/migrations/054_battery_rfid_tag.sql	new
Backend	backend/src/models/battery.model.js	findByCode OR rfid_tag; findPage SELECT; updateRfidTag()
Backend	backend/src/controllers/battery.controller.js	updateRfidTag(), byTag()
Backend	backend/src/routes/battery.routes.js	PATCH /:id/rfid-tag (office), GET /by-tag/:tag (workshop) above /:code
Backend (C)	routes/rfid.routes.js, controllers/rfid.controller.js, middlewares/reader-key.js, migration rfid_readers	new
Mobile	mobile/app.json, package.json	react-native-nfc-manager plugin + NFC permission
Mobile	mobile/src/utlis/useNfcRead.js	new hook

Layer	File	Change
Mobile	ServiceScreen.js, ClientScanScreen.js, ClientSortingScreen.js, shared/BatteryDetailScreen.js	'Tap RFID' button → existing scan handler
Mobile	mobile/src/screens/admin/ProgramTagScreen.js + navigation entry	new
Web	TechnicianHomePage.jsx, TruckVerifyModal.jsx, ReturnForm.jsx, RecycleForm.jsx, ClientBatterySortPage.jsx	accept Tab as well as Enter
Web	BatteryDetailPage.jsx, BatteriesPage.jsx	show rfid_tag; assign action; optional Tag column / No-tag filter
Web (optional)	frontend/src/utlis/use-web-nfc.js	Android Chrome Web NFC
Docs	cerebrum.md, RFID.md	update

End of document.